

Data Science Intern Interview Questions

Python, Data Science, Machine Learning, Deep Learning & Generative AI

Interview Structure

Suggested approach: Start with conceptual questions to assess fundamentals, then move to Python coding problems to evaluate problem-solving ability and Python fluency. The interviewer may select questions based on the candidate's experience and resume.

1. Python & Programming Fundamentals

1. What is the difference between a list and a tuple?
2. How do you explain mutability in Python data structures?
3. What is the difference between a shallow copy and a deep copy?
4. What is the difference between `==` and `is` in Python?
5. How do lambda functions differ from standard functions?
6. What are `*args` and `**kwargs`, and when would you use them?
7. What is the purpose of a virtual environment?
8. What is the difference between a Python module and a package?
9. How does exception handling work using `try`, `except`, `else`, and `finally`?
10. What is the purpose of the `__init__` method in a Python class?
11. How does class inheritance work in Python?
12. What is the difference between an iterator and a generator?
13. What is a Python decorator, and where might you use one?
14. What is list comprehension? How is it different from a traditional `for` loop?
15. What is the difference between `append()`, `extend()`, and `insert()` in a list?
16. How do dictionaries store and retrieve key-value pairs?
17. What is the difference between a set and a list?
18. How does Python handle memory management and garbage collection?

2. Data Science & Statistics

1. How do you define Exploratory Data Analysis (EDA)?
2. How do you handle missing values in a dataset?
3. What is the difference between correlation and covariance?
4. What is the Central Limit Theorem?
5. What is the difference between mean, median, and mode?
6. What is the difference between normalization and standardization?
7. What is an outlier, and how can you detect and handle it?
8. What is the purpose of Principal Component Analysis (PCA)?
9. What is the curse of dimensionality?
10. What is the difference between a population and a sample?
11. What is sampling bias?
12. What is the difference between descriptive and inferential statistics?

3. Machine Learning

1. What is the difference between supervised and unsupervised learning?
2. What is the difference between classification and regression?
3. What is overfitting, and how can it be prevented?
4. What is the bias-variance tradeoff?
5. What is the purpose of train, validation, and test datasets?
6. How does cross-validation help in model evaluation?
7. What is the difference between precision, recall, and F1-score?
8. How does a confusion matrix help evaluate a classification model?
9. What is the difference between bagging and boosting?
10. How does K-Means clustering determine cluster centers?
11. How does a Support Vector Machine handle non-linear data?
12. What is Gini Impurity in Decision Trees?
13. What is the difference between Random Forest and Gradient Boosting?
14. What is the purpose of hyperparameter tuning?
15. What is the difference between GridSearchCV and RandomizedSearchCV?

4. Deep Learning & Generative AI

1. What is the difference between Machine Learning and Deep Learning?
2. What is an Artificial Neural Network, and how does it work?
3. What is the role of an activation function in a neural network?
4. What is the difference between ReLU, Sigmoid, and Softmax?
5. What is backpropagation?
6. What is the purpose of an optimizer in Deep Learning?
7. What is the vanishing gradient problem?
8. What is the difference between CNNs and RNNs?
9. What is transfer learning, and why is it useful?
10. What is Generative AI, and how does it differ from traditional Machine Learning?
11. What are Large Language Models (LLMs)?
12. What is tokenization in Natural Language Processing?
13. What is the Transformer architecture?
14. What is the role of the Attention mechanism in Transformers?
15. What is prompt engineering?
16. What is Retrieval-Augmented Generation (RAG), and why is it useful?
17. What is the difference between fine-tuning and RAG?
18. What are embeddings, and how are they used in GenAI applications?
19. What is a vector database, and why is it used in RAG systems?
20. What is hallucination in LLMs, and how can it be reduced?
21. What is the difference between zero-shot, one-shot, and few-shot prompting?

5. Coding Questions

Instruction: Select one problem from a coding practice platform such as GeeksforGeeks and ask the candidate to code it during the interview. Alternatively, choose one or more problems from the list below. Focus on the candidate's approach, edge-case handling, code quality, and ability to explain their solution.

Basic Level

- 1. Fibonacci Series:** Write a function to print the first N Fibonacci numbers. Ask the candidate to explain the time and space complexity.
- 2. Palindrome Number:** Write a function to check whether a given number is a palindrome without converting the number to a string.
- 3. Armstrong Number:** Write a function to check whether a given number is an Armstrong number.
- 4. Prime Number:** Write a function to check whether a given number is prime.
- 5. Factorial:** Write a function to calculate the factorial of a number using both iteration and recursion.
- 6. Reverse a String:** Write a function to reverse a string without using Python's built-in `reversed()` function.
- 7. Count Vowels:** Write a function that counts the number of vowels in a given string.

8. Find Maximum and Minimum: Given a list of numbers, find the maximum and minimum values without using `max()` or `min()`.

9. Sum of Digits: Write a function that calculates the sum of all digits in a given integer.

10. FizzBuzz: Print numbers from 1 to N. For multiples of 3 print Fizz, for multiples of 5 print Buzz, and for multiples of both print FizzBuzz.

Medium Level

11. Two Sum: Given a list of integers and a target value, return the indices of two numbers that add up to the target.

12. Remove Duplicates: Given a list, remove duplicate elements while preserving the original order.

13. Find the Second Largest: Find the second-largest distinct number in a list without sorting the entire list.

14. Anagram Check: Write a function to determine whether two strings are anagrams of each other.

15. Character Frequency: Given a string, return a dictionary containing the frequency of each character.

16. First Non-Repeating Character: Find the first character in a string that does not repeat.

17. Merge Two Sorted Lists: Given two sorted lists, merge them into one sorted list without using the built-in `sort()` function.

18. Move Zeros to the End: Move all zeros in a list to the end while maintaining the relative order of non-zero elements.

19. Missing Number: Given an array containing N distinct numbers from 0 to N, find the missing number.

20. Word Frequency: Given a paragraph, count the frequency of each word after handling case differences and basic punctuation.

Python Fluency & Data Handling Challenges

21. Dictionary-Based Grouping: Given a list of dictionaries containing employee information, group employees by department using Python.

22. List Comprehension: Create a diamond pattern or generate a filtered and transformed list using a single-line nested list comprehension.

23. String Template Formatter: Write a function that accepts a template containing placeholders such as `{name}` and `{role}`, and replaces them using values from a dictionary.

24. Safe Type Conversion: Write a function that accepts a string, converts it to a float, and raises a descriptive `ValueError` if the conversion fails.

25. Data Cleaning Function: Given a list of strings containing missing values, extra whitespace, and inconsistent casing, write a function to clean and standardize the data.

26. Simple Pandas Task: Given a `DataFrame` containing sales data, calculate total sales by category and identify the category with the highest sales.

6. Suggested Interview Evaluation

Area	What to Evaluate
Python Fundamentals	Syntax, data structures, functions, OOP, exceptions, comprehensions
Problem Solving	Ability to break a problem into smaller steps and identify edge cases
Coding Fluency	Writing clean, readable, and logically correct Python code
Data Science	Statistics, EDA, preprocessing, and data interpretation
Machine Learning	Algorithms, evaluation metrics, overfitting, and model selection
Deep Learning	Neural networks, optimization, CNN/RNN basics
GenAI	LLMs, Transformers, embeddings, RAG, prompting, and basic AI agents
Communication	Ability to explain the approach and reasoning clearly

Recommended coding progression: Start with one Basic problem such as Fibonacci, Palindrome, Prime, or Armstrong Number. If the candidate solves it comfortably, move to a Medium problem such as Two Sum, First Non-Repeating Character, or Merge Two Sorted Lists. For a Data Science Intern role, finish with a small Pandas/data manipulation problem.